FIREWALL LOG ANOMALY DETECTOR

An Unsupervised Machine Learning Approach to Identifying Anomalous Network Traffic

PROJECT REPORT

Technology Stack: Python · Pandas · Scikit-learn · Google Colab

Machine Learning Algorithm: Isolation Forest

Prepared by:

Hawa Doumbe Traore

Academic Year:

2026–2027

Table of Contents

Table of Contents ---------- 2

GLOSSARY ---------- 7

LIST OF ABBREVIATIONS ---------- 11

GENERAL INTRODUCTION ---------- 12

1. Background and Context: ---------- 12

2. Project Objectives ---------- 12

3. Methodology and Scope ---------- 13

4. Organization of the Report ---------- 13

Part I — Theoretical Part ---------- 14

1.Introduction ---------- 15

1.1 Cybersecurity and Network Monitoring: ---------- 15

1.2 Firewall Logs and Network Anomalies: ---------- 15

1.3 Problem Statement: ---------- 16

1.4 Project Objectives: ---------- 16

1.5 Scope of the Project: ---------- 17

2.Firewall Logs and Network Anomalies: ---------- 18

2.1 The Role of a Firewall: ---------- 18

2.2 Structure of Firewall Logs: ---------- 18

2.3 Normal and Potentially Anomalous Activity: ---------- 20

2.4 Examples of Network Anomalies: ---------- 20

2.5 The Importance of Context: ---------- 21

3.Anomaly Detection Techniques ---------- 22

3.1 Definition of Anomaly Detection: ---------- 22

3.2 Signature-Based Detection: ---------- 22
3.3 Rule-Based Detection: ---------- 23
3.4 Machine-Learning-Based Anomaly Detection: ---------- 24
3.5 Supervised and Unsupervised Learning: ---------- 24
3.6 The Isolation Forest Algorithm: ---------- 25
3.7 Contamination Parameter: ---------- 26
3.8 Standardization of Numerical Features: ---------- 27
3.9 Comparison of Detection Approaches: ---------- 27
4.Detection Methodology ---------- 28
4.1 Overview of Methodology: ---------- 28
4.2 Dataset Generation and Collection: ---------- 29
4.3 Data Preprocessing: ---------- 30
4.4 Feature Engineering: ---------- 30
4.5 Feature Scaling: ---------- 31
4.6 Anomaly Detection with Isolation Forest: ---------- 32
4.7 Risk Scoring and Classification: ---------- 32
4.8 Result Analysis: ---------- 34
5.Features Used for Detection ---------- 34
5.1 Definition of Feature: ---------- 34
5.2 Original Data Fields: ---------- 35
5.3 Derived Features: ---------- 36
5.4 Combining Features: ---------- 37
5.5 Feature Selection and Implementation Consistency: ---------- 37
6.Theoretical System Architecture ---------- 38

6.1 Architecture Overview: ---------- 38
6.2 Data Input: ---------- 38
6.3 Data Preparation and Feature Engineering: ---------- 38
6.4 Feature Standardization: ---------- 39
6.5 Anomaly Detection: ---------- 39
6.6 Risk Scoring: ---------- 39
6.7 Results and Visualization: ---------- 40
6.8 Overall Architecture Diagram: ---------- 40
7.Technologies and Tools ---------- 41
7.1 Python: ---------- 41
7.2 Pandas: ---------- 42
7.3 NumPy: ---------- 42
7.4 Scikit-learn: ---------- 42
7.5 Matplotlib: ---------- 43
7.6 Seaborn: ---------- 43
7.7 Google Colab: ---------- 43
7.8 Summary of Technologies: ---------- 43
8. Conclusion ---------- 45
Part II — Practical Part ---------- 46
1. Introduction ---------- 47
2. Development Environment and Dataset Preparation ---------- 47
a. Development environment and libraries: ---------- 47
b. Generating synthetic firewall records: ---------- 48
c. Adding timestamps and IP addresses: ---------- 50

3. Exploratory Data Analysis ---------- 51
a. Inspecting the dataset: ---------- 51
b. Data quality and traffic distributions: ---------- 53
4. Feature Engineering and Preproces---------- 58
a. Creating derived features: ---------- 58
b. Selecting model inputs and scaling: ---------- 59
5. Training and Applying the Isolation Forest Model ---------- 60
a. Model training: ---------- 60
b. Predicting and summarizing anomalies: ---------- 61
6. Anomaly Review, Risk Scoring, and Alert Generation ---------- 63
a. Inspecting detected records: ---------- 63
b. Exploring anomaly patterns: ---------- 64
c. Rule-based risk score: ---------- 66
d. Assigning risk levels: ---------- 68
e. Generating security alerts: ---------- 69
7. Functional Testing and Exporting the Results ---------- 71
a. Reusable function for new records: ---------- 71
b. Testing two simulated cases: ---------- 73
c. Saving the model and processed dataset: ---------- 74
8. Summary and Limitations ---------- 77
a. Summary of results: ---------- 77
b. Limitations: ---------- 78
9. Conclusion and future improvements ---------- 79
List of Figures: ---------- 80

List of Tables: ---------- 82

References: ---------- 83

General conclusion---------- 84

Glosarry:

Anomaly: An observation or network event that differs significantly from the expected pattern of normal activity. An anomaly may indicate suspicious behaviors, but it does not necessarily represent a confirmed cyberattack.

Anomaly Detection: The process of identifying unusual patterns or observations within a dataset that may require further investigation.

Anomaly Label: A textual classification assigned to a network record according to the model's prediction. In this project, the labels are *Normal* and *Anomaly*.

Anomaly Prediction: The output produced by the Isolation Forest model. In this project, a prediction of `1` indicates a record classified as normal, while `-1` indicates a record classified as an anomaly.

Artificial Intelligence (AI): A field of computer science concerned with developing systems capable of performing tasks that typically require human intelligence, such as recognising patterns, learning from data, and making predictions.

Bytes per Packet: An engineered feature calculated by dividing the total number of bytes by the number of packets. It estimates the average data volume per packet.

Classification: The process of assigning an observation to a category based on defined rules or a model's output. In this project, model predictions identify normal or anomalous records, while a separate rule-based process assigns risk categories.

Contamination: A parameter in the Isolation Forest algorithm that specifies the expected proportion of outliers in a dataset. It was set to `0.05` (5%) in this project. It is a modelling parameter, not a measure of model accuracy or proof of the real-world proportion of malicious traffic.

Cybersecurity: The practice of protecting computer systems, networks, applications, and data against unauthorised access, attacks, damage, and other digital threats.

Dataset: A structured collection of data organised into records and variables. The dataset used in this project contains 5,250 synthetically generated firewall log records.

DataFrame: A two-dimensional data structure provided by the Pandas library, organised into rows and columns. In this project, the main dataset is stored in a DataFrame named `df`.

Data Preprocessing: The preparation of data before it is used by a machine learning model. It may include inspecting data quality, selecting features, transforming variables, and standardising numerical values.

Exploratory Data Analysis (EDA): The process of examining and visualising a dataset to understand its structure, distributions, relationships, and potential irregularities before applying machine learning techniques.

Feature: An individual variable used to describe an observation and provide information to a machine learning model. Examples in this project include source port, destination port, duration, packet count, byte count, and connection rate.

Feature Engineering: The process of creating or transforming variables to make them more useful for machine learning. In this project, engineered features include `hour`, `bytes_per_packet`, `packet_rate`, `failed_connection_ratio`, and `unusual_port`.

Feature Matrix: A structured table containing the selected input variables used by a machine learning model. The feature matrix in this project contains 5,250 records and 12 features.

Firewall: A network security mechanism that monitors and controls incoming and outgoing network traffic according to defined security rules.

Firewall Log: A record containing information about network traffic or connection events, such as IP addresses, port numbers, protocols, packet counts, and connection statistics. Firewall logs can support security monitoring and the investigation of unusual network activity.

Isolation Forest: An unsupervised machine learning algorithm designed to identify anomalies by isolating observations through repeated random partitioning. Observations that are easier to isolate are more likely to be classified as anomalous.

Machine Learning (ML): A branch of artificial intelligence that enables computer systems to learn patterns from data and use them to make predictions or identify structures without requiring every decision to be explicitly programmed.

Model Training: The process through which a machine learning algorithm analyses data to construct a model that can subsequently generate predictions. In this project, Isolation Forest is trained using firewall-related features without requiring pre-labelled examples of attacks.

Model Inference: The process of applying a trained model to new input data to produce a prediction or classification.

Normal Traffic: Network activity classified as normal by the model based on the patterns learned from the dataset. This classification does not independently prove that the traffic is safe.

Outlier: An observation that differs substantially from the majority of the observations in a dataset. An outlier may reflect unusual but legitimate activity or potentially suspicious behaviour.

Packet Rate: An engineered feature calculated by dividing the number of packets by the connection duration. The implementation includes a safeguard against division by zero.

Preprocessing: The preparation and transformation of raw data before analysis or model application. In this project, it includes feature selection, feature engineering, and standardisation.

Protocol: A set of rules that governs communication between devices across a network. TCP and UDP are examples of transport-layer protocols represented in the generated dataset.

Risk Classification: The assignment of a risk category to a network record based on its calculated risk score. This project uses four categories: Low, Medium, High, and Critical.

Risk Score: A numerical score calculated using predefined rules and indicators. In this project, points are assigned for model-detected anomalies, high failed-connection counts, high connection rates, and unusual destination ports. The score is capped at 100 and is not a calibrated probability of an attack.

Security Alert: A message generated to highlight a network record that meets the project's risk criteria. The alert text reflects the assigned risk category and is intended to support further investigation.

StandardScaler: A preprocessing class from Scikit-learn that standardises numerical features using the mean and standard deviation calculated during fitting. The fitted scaler is reused when transforming new records.

Standardisation: A preprocessing technique that transforms numerical features to a common scale, generally by centring values around a mean of zero and scaling them according to their standard deviation. It helps prevent variables with large numerical ranges from dominating simply because of their scale.

Synthetic Data: Artificially generated data designed to reproduce selected characteristics of real-world observations. The firewall logs used in this project are synthetic and were not collected from a production firewall.

Unsupervised Learning: A machine learning approach that identifies patterns, structures, or unusual observations in data without requiring predefined labels for every observation during model training.

Unusual Destination Port: A destination port that does not belong to the predefined list of common ports used by this project. It is treated as an indicator for feature engineering and risk scoring, but it does not automatically mean that the traffic is malicious.

Validation and Testing: The process of examining a system's behaviour against selected test cases or evaluation criteria. In this project, two simulated connection examples were tested to check basic detection functionality. These tests do not establish the model's general performance on real-world network traffic.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
IP	Internet Protocol
ML	Machine Learning
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

GENERAL INTRODUCTION

1. Background and Context:

The increasing use of computer networks and digital services has made network security an important concern for organizations and individuals. Networks continuously generate information about communication activities, including connection attempts, traffic volume, destination ports, and data transmission patterns. These events can be recorded in firewall logs, which provide useful information for monitoring network activity and investigating potentially suspicious behavior.

However, analyzing large volumes of network logs manually can be time-consuming and may make unusual patterns difficult to identify. Machine learning offers techniques that can help analyze network data and detect observations that differ from expected behavior. Among these techniques, anomaly detection is particularly useful when predefined labels identifying every possible threat are not available.

2. Project Objectives

The main objective of this project is to develop a Python-based **Firewall Log Anomaly Detector** using an unsupervised machine learning approach. The project applies the Isolation Forest algorithm to identify potentially unusual patterns in firewall-related data.

The specific objectives are to generate and examine a dataset representing network traffic, perform feature engineering and data preprocessing, train an Isolation Forest model, identify anomalous records, and implement a rule-based risk-scoring mechanism to help prioritize events for further investigation. The system also generates security alert messages, provides a

reusable detection function, and exports the model and analysis results for future use.

3. Methodology and Scope

The implementation was developed in Google Colab using Python and libraries including Pandas, NumPy, Matplotlib, and Scikit-learn. A synthetic dataset containing 5,250 firewall log records was created to simulate selected characteristics of normal and potentially suspicious network activity. Twelve features were selected for model training following exploratory data analysis and preprocessing.

The trained model was then used to classify records as normal or anomalous. A separate rule-based mechanism assigned risk scores and categories based on selected indicators, while test cases were used to examine the basic behavior of the detection function.

The project is a proof of concept developed using synthetic data. Therefore, its results demonstrate the implementation of the proposed approach but do not establish its effectiveness on real firewall logs or confirm the detection of actual cyberattacks.

4. Organization of the Report

This report is divided into two main parts. The first part presents the theoretical foundations of firewall log analysis, anomaly detection, unsupervised machine learning, and the Isolation Forest algorithm. The second part describes the practical implementation, including dataset generation, exploratory data analysis, feature engineering, model training, anomaly analysis, risk scoring, security alert generation, testing, and the export of results.

Part I — Theoretical Part

1. Introduction

1.1 Cybersecurity and Network Monitoring:

Cybersecurity refers to the protection of computer systems, networks, applications, and data against unauthorized access, damage, disruption, and other digital threats. As organizations increasingly rely on interconnected technologies, monitoring network activity has become an important component of information security.

Network monitoring involves observing and analyzing network events to understand communication patterns, identify operational problems, and detect potentially suspicious behavior. It can rely on information collected from network devices, servers, applications, and security systems.

One important source of network information is the firewall. In addition to controlling network traffic, firewalls can generate logs that record connection-related events. These records provide useful information for understanding network activity and investigating potential security incidents.

1.2 Firewall Logs and Network Anomalies:

A firewall log is a record of a network event observed or processed by a firewall. Depending on the logging configuration, an entry may contain a timestamp, source and destination IP addresses, port numbers, network protocol, traffic volume, and other connection-related information.

Although these records are useful for security monitoring, large volumes of logs can be difficult to examine manually. Suspicious behavior may also become apparent only when several records are analyzed together.

A network anomaly is an event or behavior that differs significantly from an expected pattern. Examples include an unusually high connection rate, repeated connection attempts, or communication involving an unexpected number of ports. Such behavior may indicate a security problem, but it can also result from legitimate activity or configuration errors.

Automated anomaly detection can help analyze firewall logs systematically and identify records that deserve further investigation.

1.3 Problem Statement:

The main problem addressed by this project is the identification of potentially anomalous network activity within firewall log data.

Raw firewall records contain information that can be difficult to interpret when examined individually or in large quantities. Manually identifying unusual patterns can be time-consuming, particularly when the data contains thousands of entries.

The project addresses this challenge by developing a Python-based system that processes firewall log data, extracts relevant numerical features, applies an anomaly-detection algorithm, and assigns risk levels to the resulting observations.

The system is intended to support network monitoring by highlighting potentially unusual records. It does not independently establish whether a detected event represents a confirmed cyberattack.

1.4 Project Objectives:

The main objective is to develop a firewall log anomaly detector capable of identifying potentially unusual network activity through automated data analysis. The specific objectives are to:

- Generate and organize firewall log data in a structured format.
- Prepare the data for numerical analysis.
- Extract relevant features from network events.
- Apply an unsupervised machine-learning algorithm to identify anomalous observations.
- Assign risk scores and corresponding risk levels using predefined criteria.
- Summarize and visualize the detection results.
- Demonstrate how Python and machine learning can support cybersecurity monitoring.

1.5 Scope of the Project:

The project focuses on analyzing a generated firewall log dataset using Python, data-processing libraries, and the Isolation Forest algorithm. The implementation combines machine-learning-based anomaly detection with a rule-based risk-scoring mechanism.

The project demonstrates the fundamental stages of a log-analysis pipeline, from data preparation for anomaly identification and risk classification. It is a practical prototype rather than a production-ready intrusion detection system.

In particular, the results depend on the characteristics of the generated dataset, the selected features, the model configuration, and the risk-scoring thresholds. Further validation against representative real-world firewall logs would be necessary before using the system in an operational network.

2. Firewall Logs and Network Anomalies:

2.1 The Role of a Firewall:

A firewall is a security mechanism that monitors and controls network traffic according to configured security policies. It can allow, reject, or block communication depending on the applicable rules and the characteristics of the traffic.

Firewalls are commonly positioned between networks with different trust levels, such as an internal organizational network and the public Internet. They may also protect individual systems or specific network segments.

In addition to controlling communication, many firewalls record information about network events. These records provide a basis for examining connection activity and investigating potential security incidents.

2.2 Structure of Firewall Logs:

The exact structure of a firewall log depends on the device, software, and logging configuration. Common fields include the following:

Field	Description
Timestamp	Indicates when the event occurred.
Source IP address	Identifies the originating host or network interface.
Destination IP address	Identifies the target host or service.

Source port	Identifies the source-side communication port.
Destination port	Indicates the port targeted by the connection.
Protocol	Specifies the network protocol, such as TCP, UDP, or ICMP.
Duration	Represents the recorded duration of a connection when available.
Packet count	Indicates the number of packets associated with an event.
Byte count	Represents the volume of data transferred.
Failed connections	Records or represents failed connection attempts, depending on the dataset.
Connection rate	Represents the frequency of connections over a defined interval.

Table 1: Description of raw firewall log attributes and fields

These fields do not necessarily appear in every firewall log. Their availability depends on the data source and how the records are generated.

For this project, the structured dataset contains timestamp, source and destination IP addresses, source and destination ports, protocol, duration, packet count, byte count, failed connections, and connection rate. These fields provide information used to construct the analytical features.

2.3 Normal and Potentially Anomalous Activity:

Normal network activity is communication that is consistent with the expected use of a system or network. Examples include accessing authorized web services, making DNS requests, and communicating with designated application servers.

Potentially anomalous activity differs from the patterns represented in the data. Examples include:

- A sudden increase in the number of connections.
- An unusually high connection rate.
- Repeated failed connection attempts.
- Communication involving an unexpectedly large number of ports.
- An unusual combination of connection frequency and traffic volume.

For example, a device that normally generates a modest number of connections but suddenly generates thousands may require investigation. The change could result from automated scanning, a malfunctioning application, a legitimate workload increase, or another cause.

Consequently, abnormal behavior should be treated as an indicator for investigation rather than conclusive evidence of malicious activity.

2.4 Examples of Network Anomalies:

- **Repeated connection attempts:** A host may repeatedly attempt to establish connections with a particular service. When these attempts occur at an unusually high frequency, they may indicate a configuration problem, automated scanning, or an unauthorized access attempt.

- **Port-scanning patterns:** Port scanning involves attempting to communicate with multiple ports on a target system to discover available services. A large number of connections to different ports within a short period may represent a potentially suspicious pattern.

- **Unusual connection rates:** A sudden increase in connection frequency may indicate automated activity, software malfunction, or an unusually high workload. The significance of the increase depends on the system's normal behavior and operational context.

- **Unusual traffic characteristics:** Changes in packet counts, byte counts, connection duration, or other measurable characteristics may indicate behavior that differs from the patterns represented in the dataset.

These examples illustrate why examining several characteristics together can provide more useful information than relying on a single field.

2.5 The Importance of Context:

An event cannot always be classified as suspicious based on a universal rule. For example, a high connection rate may be normal for a busy server but unusual for a workstation. Similarly, activity involving a particular port may be legitimate in one environment and unexpected in another.

The interpretation of an anomaly therefore depends on the characteristics of the environment, the quality of the data, and the detection method.

The project uses an algorithmic approach to identify observations that differ from patterns in the dataset. Its output should be interpreted as a preliminary indication of unusual behavior, not as a definitive security diagnosis.

3. Anomaly Detection Techniques

3.1 Definition of Anomaly Detection:

Anomaly detection is the process of identifying observations or patterns that differ significantly from the expected characteristics of a dataset.

In cybersecurity, anomaly detection can be applied to network traffic, authentication events, system activity, and firewall logs. It helps identify behavior that may warrant further examination.

The general process consists of preparing data, identifying relevant characteristics, applying a detection method, and examining the resulting observations.

Anomaly detection can be implemented through predefined rules, statistical methods, or machine-learning algorithms. Each approach has different requirements and limitations.

3.2 Signature-Based Detection:

Signature-based detection identifies known threats by comparing observed activity with previously defined signatures or patterns.

A signature may represent a recognizable characteristic of a known attack. When observed activity matches the signature, the system can generate an alert.

This approach can be effective for detecting known threats with clearly defined patterns. However, it depends on the availability and quality of the signatures. New or modified threats may not be detected if their behavior does not match an existing signature.

Signature-based detection is therefore useful for recognizing known attack patterns but may require complementary techniques to identify other forms of suspicious activity.

3.3 Rule-Based Detection:

Rule-based detection applies to predefined conditions to determine whether an event should be flagged.

For example, a system may assign additional risk points when the number of failed connection attempts exceeds a specified threshold. Another rule may increase the risk score when the connection rate becomes unusually high.

The general process is:

Input data → Evaluate predefined conditions → Apply rules → Produce a result

Rule-based systems are relatively easy to explain and implement. Their behavior is determined by explicit conditions, making it possible to understand why a particular observation received a given score.

However, their effectiveness depends on the quality of the rules and thresholds. A threshold that is appropriate for one environment may be unsuitable for another. Rules may therefore require adjustment as network conditions change.

In this project, predefined rules are used to assign risk points after the anomaly detection stage. This risk-scoring mechanism complements the machine-learning output rather than replacing it.

3.4 Machine-Learning-Based Anomaly Detection:

Machine learning provides methods for identifying patterns in data and using those patterns to make predictions or detect unusual observations.

In network security, machine-learning algorithms can analyze measurable characteristics of network activity, including connection rates, traffic volumes, port information, and other numerical features.

The appropriate algorithm depends on the data available, the objective of the analysis, and whether reliable labels are provided.

Machine-learning-based anomaly detection is useful when the goal is to identify observations that differ from the broader data distribution. However, detected anomalies are not necessarily malicious, and model performance depends on the quality and representativeness of the data.

3.5 Supervised and Unsupervised Learning:

Supervised learning:

Supervised learning uses labeled examples to learn about a relationship between input features and known outcomes. In a cybersecurity application, labels might distinguish normal traffic from confirmed malicious activity.

The model is trained using the labeled examples and subsequently evaluated on separate data.

A limitation of supervised learning is that accurate labels may be difficult to obtain. Furthermore, a model trained on particular attack types may not generalize well to unfamiliar threats.

Unsupervised learning:

Unsupervised learning analyzes data without requiring predefined target labels for every observation. Depending on the algorithm, it can identify groups, structures, or observations that differ from the broader data distribution.

This approach is useful for exploratory anomaly detection because the system can identify unusual observations without first requiring each record to be labeled as normal or malicious.

The Firewall Log Anomaly Detector uses Isolation Forest, an unsupervised anomaly detection algorithm. It identifies observations that can be isolated relatively easily within randomly constructed trees. Such observations tend to receive greater anomaly scores than observations that require more partitioning to isolate.

3.6 The Isolation Forest Algorithm:

Isolation Forest was introduced by Liu, Ting, and Zhou (2008). It is designed to identify anomalies by isolating observations rather than constructing an explicit model of normal behavior.

The algorithm builds an ensemble of random isolation trees. At each stage, a feature and a split value are selected to divide the data. Observations that differ substantially from the majority may be isolated after relatively few divisions.

The underlying principle is that unusual observations are often easier to isolate than typical observations.

The main stages can be summarized as follows:

1. Receive a dataset containing numerical features.

2. Construct isolation of trees using random feature selections and split values.
3. Determine how easily each observation is isolated.
4. Calculate anomaly-related scores from the isolation process.
5. Identify observations classified as outliers according to the model's decision threshold.

Isolation Forest is suitable for this project because it can analyze multiple numerical characteristics without requiring manually supplied labels for every record.

However, its predictions depend on the input features, model configuration, and data distribution. A model-generated anomaly label should not automatically be interpreted as proof of an attack.

3.7 The Contamination Parameter:

Isolation Forest includes a parameter called `contamination`, which helps determine the threshold used to classify observations as anomalies.

In this project, the model is configured with `contamination=0.05`, representing an assumed anomaly proportion of approximately 5% for threshold selection.

This setting is a model configuration choice. It does not establish that exactly 5% of all real-world firewall events are malicious.

The actual number of observations flagged by the model depends on the dataset, the algorithm, and the decision threshold. The resulting anomaly proportion should therefore be interpreted in the context of the model configuration and the data being analyzed.

3.8 Standardization of Numerical Features:

The implementation uses `StandardScaler` from scikit-learn to standardize numerical features before anomaly detection.

Standardization transforms a numerical feature by subtracting its mean and dividing its standard deviation, calculated from the data used to fit the scaler. This places features on a comparable scale.

For example, byte counts may have much larger numerical values than ratios or rates. Standardization can make the numerical representation more consistent across these variables.

Although Isolation Forest does not generally require feature scaling in the same way as distance-based algorithms, the scaler is part of this project's processing pipeline. Its use should be documented as an implementation choice rather than a universal requirement of Isolation Forest.

3.9 Comparison of Detection Approaches:

Characteristic	Signature-based	Rule-based	Isolation Forest
Main principle	Match known patterns	Evaluate predefined conditions	Isolate unusual observations
Requires known attack signatures	Yes, for signature matches	No	No

Requires labeled training data	Not necessarily	No	No
Adapts automatically to new data	Only with updated signatures	No, unless rules change	Learns structure from the fitted dataset
Interpretability	High for known matches	High	Requires additional explanation
Main limitation	May miss unknown patterns	Depends on rule quality	May flag legitimate unusual activity

Table 2: Comparative analysis of signature-based, rule-based, and Isolation Forest detection approaches

These approaches serve different purposes and are not mutually exclusive. The project combines an unsupervised machine-learning method with a predefined risk-scoring system to provide both anomaly identification and a more interpretable risk category.

4. Detection Methodology

4.1 Overview of the Methodology:

The methodology describes how the Firewall Log Anomaly Detector transforms structured firewall data into anomaly labels and risk classifications.

The implementation follows this general sequence:

Dataset generation → Data preparation → Feature engineering → Feature scaling → Isolation Forest → Risk scoring → Result analysis and visualization

Each stage contributes to the overall process.

4.2 Dataset Generation and Collection:

The project uses a generated firewall log dataset containing 5,250 records. The dataset is created for experimentation and contains simulated normal and anomalous activity.

The intended dataset composition includes approximately 5,000 normal records and 250 anomalous records. The generated observations provide a controlled environment in which the detection workflow can be demonstrated.

The dataset contains the following fields:

- `timestamp`
- `src_ip`
- `dst_ip`
- `src_port`
- `dst_port`
- `protocol`
- `duration`
- `packets`
- `bytes`
- `failed_connections`
- `connection_rate`

The quality and design of the generated data influence the results. Because the records are simulated, successful detection within this dataset does not by itself demonstrate equivalent performance on real network traffic.

4.3 Data Preprocessing:

Data preprocessing prepares the dataset for analysis. It involves checking the data structure, ensuring that numerical fields are represented appropriately, and preparing the information required for feature engineering.

Depending on the implementation, preprocessing may include converting timestamps, checking numerical data types, and ensuring that the selected features are suitable for the model.

The objective is to produce a consistent numerical representation of network activity while preserving the information needed for anomaly detection.

4.4 Feature Engineering:

Feature engineering involves transforming available data into variables that can provide useful information to the detection algorithm.

In addition to the original fields, the project derives features including:

Feature	Purpose
`hour`	Represents the hour extracted from the timestamp.
`bytes_per_packet`	Relates the recorded byte count to the packet count.

`packet_rate`	Represents packet activity relative to the relevant time measure used in the implementation.
`failed_connection_ratio`	Represents the proportion of failed connections according to the implemented calculation.
`unusual_port`	Encodes whether a port satisfies the project's predefined unusual port condition.

Table 3: Summary and purpose of engineered features derived from raw firewall logs

These features provide additional numerical representations of network activity.

The exact formulas, handling of zero values, and thresholds must follow the implemented code. In particular, the `unusual port` feature reflects a predefined criterion; it does not mean that every non-standard port is inherently malicious.

4.5 Feature Scaling:

After feature engineering, the selected numerical features are standardized using `StandardScaler`.

The scaler transforms the features into a standardized representation based on the mean and standard deviation calculated during fitting. The resulting data is then supplied to the Isolation Forest model.

In a production deployment, the scaler should be fitted on appropriate training or baseline data and reused consistently when processing subsequent records. This helps prevent inconsistencies between the data used to prepare the model and the data evaluated later.

4.6 Anomaly Detection with Isolation Forest:

The prepared features are passed to the Isolation Forest model, configured with 200 estimators, a contamination parameter of 0.05, and a random state of 42.

The model uses an ensemble of isolation trees to identify observations that are easier to isolate than typical observations in the dataset.

The model's predictions distinguish observations classified as normal from those classified as anomalies. The output is a model-based assessment of unusualness, not a verified determination that an observation represents malicious activity.

The use of a fixed random state supports reproducibility under the same software environment and input conditions.

4.7 Risk Scoring and Classification:

Following anomaly detection, the implementation applies to a separate rule-based risk-scoring mechanism. This stage assigns points according to the model's anomaly output and selected indicators of potentially risky behavior.

The scoring logic is defined as follows:

Condition	Risk points

Observation classified as anomalous by Isolation Forest	+50
Failed connections exceed 10	+20
Connection rate exceeds 50	+15
Unusual-port condition is satisfied	+15

Table 4: Rule-based risk scoring conditions and assigned risk points

The total score is capped at 100.

The final risk level is assigned according to the following thresholds:

Risk score	Classification
80–100	Critical
60–79	High
30–59	Medium
0–29	Low

Table 5: Risk score thresholds and threat level classifications

This mechanism combines the machine-learning prediction with predefined rules. Isolation Forest determines whether an observation is classified as an anomaly, while the scoring rules determine the risk points and final risk category.

The resulting risk score should be understood as a project-specific indicator rather than a calibrated probability that a cyberattack is occurring. The thresholds are design choices and would require validation before being used for operational security decisions.

4.8 Result Analysis:

The system produces summaries of the observations classified as normal or anomalous and their associated risk levels. These results can be examined through numerical summaries, tables, and visualizations.

Useful indicators include the total number of records, the number classified as anomalous, the percentage of anomalies, and the distribution of risk categories.

The results can help demonstrate how the different stages of the pipeline operate together. However, evaluating detection quality requires more than counting flagged records. If reliable ground-truth labels are available, measures such as precision, recall, and F1-score can help determine how well the model distinguishes known anomalous observations from normal ones.

Because this project uses simulated data, any performance assessment should be presented within the limitations of that dataset.

5. Features Used for Detection

5.1 Definition of a Feature:

A feature is a measurable characteristic used by an analytical method to represent an observation. In firewall log analysis, features can come directly from the original records or be derived from existing fields.

The choice of features matters because the anomaly-detection algorithm operates on the numerical information supplied to it. Relevant features can help represent differences in connection behavior, traffic volume, and other network characteristics.

5.2 Original Data Fields:

The project's original fields provide the basis for feature engineering.

IP addresses: The source and destination IP addresses identify the endpoints involved in network communication. They provide useful context for grouping and interpreting records. However, IP addresses are not automatically meaningful for numerical measurements of network behavior. Their use as model inputs depends on how they are encoded or transformed. The model's selected feature list should therefore be distinguished from the complete list of fields in the dataset.

Ports: Source and destination ports identify communication endpoints associated with network services. Destination-port information can help describe which services are being contacted. The project's `unusual_port` feature converts a predefined port-related condition into a feature suitable for the scoring logic and, where included in the selected feature set, numerical analysis.

Protocol: The protocol field identifies the communication protocol, such as TCP or UDP. Protocol information provides context about the type of communication represented by each record. If a categorical protocol field is used as a model input, it must be encoded numerically or transformed into an appropriate representation.

Timestamp: the timestamp indicates when a network event occurred. In this project, the hour feature is derived from the timestamp to represent the hour associated with an event. Temporal information can help describe when

activity occurs, although an hour of value alone does not establish whether the activity is abnormal.

Duration, packets, and bytes: Duration, packet count, and byte count provide information about connection activity and traffic volume. These values can be used directly or combined to calculate additional features. Their interpretation depends on the units, definitions, and quality of the underlying data.

Failed connections and connection rate: Failed connection information and connection rate provide indicators of connection behavior. High values may contribute to the identification or risk assessment of unusual activity, depending on the conditions implemented by the project.These indicators should not be treated as proof of malicious activity without additional evidence.

5.3 Derived Features:

The project derives additional features from the original data:

Hour: Extracted from the timestamp to represent the hour associated with an event.

Bytes per packet: Relates the recorded byte count to the packet count. Its calculation must account for records in which the packet count is zero.

Packet rate: Represents packet activity relative to the time measure used in the implementation. The denominator must be clearly defined in the code and documentation.

Failed-connection ratio: Represents failed connections relative to the relevant total or denominator used in the calculation. The report should use the exact formula implemented by the program.

Unusual-port indicator: Encodes whether a port satisfies the project's predefined condition. The meaning of this indicator depends on the configured port rule.

Derived features can make patterns more explicit, but they also introduce assumptions. Their formulas and thresholds should be documented and applied consistently.

5.4 Combining Features:

An individual feature rarely provides enough context to establish that an event is malicious. A high connection rate, for example, may occur during legitimate periods of heavy activity.

Combining multiple characteristics can provide a richer representation of the observation. In this project, the Isolation Forest model analyzes the selected numerical features, while the risk-scoring mechanism considers the model's anomaly label and predefined conditions involving failed connections, connection rate, and unusual ports.

The two stages have different roles: the model identifies observations that differ from the dataset's patterns, and the scoring mechanism translates selected indicators into a risk category.

5.5 Feature Selection and Implementation Consistency:

The final list of model inputs must correspond to the features actually supplied to Isolation Forest. A field may exist in the dataset without being used by the model.

Similarly, a derived feature may be used by the risk-scoring mechanism without necessarily being included in the model's input matrix.

Documenting these distinctions is important for reproducibility. It allows readers to understand which data the algorithm analyzes, how the additional risk score is calculated, and what information contributes to the final classification.

6. Theoretical System Architecture

6.1 Architecture Overview:

The system architecture describes the sequence through which firewall log data is transformed into anomaly classifications and risk levels.

The architecture of the project consists of the following stages: **Generated Firewall Logs → Data Preparation → Feature Engineering → Standardization → Isolation Forest → Rule-Based Risk Scoring → Results and Visualization**

Each stage has a defined responsibility and contributes to the final output.

6.2 Data Input:

The data-input stage provides the structured firewall log dataset. Each record contains information about a simulated network event, including connection-related fields and numerical indicators.

The dataset serves as the input for all subsequent processing and detection operations.

6.3 Data Preparation and Feature Engineering:

The data-preparation stage checks and organizes records for analysis. Feature engineering then derives additional variables, including hour, bytes per packet, packet rate, failed-connection ratio, and the unusual-port indicator.

These operations transform the original data into a representation suitable for numerical processing.

6.4 Feature Standardization:

The selected numerical features are standardized using `StandardScaler`. This stage produces the numerical matrix passed to the anomaly detection model.

The fitted scaler and feature definitions must remain consistent when the system is used to evaluate additional records.

6.5 Anomaly Detection:

The Isolation Forest component analyzes the selected features and classifies observations as normal or anomalous according to its decision threshold.

This stage provides the machine-learning output used by the subsequent risk-scoring component.

6.6 Risk Scoring:

The risk-scoring component applies the predefined rules to calculate a score between 0 and 100. The score incorporates the model's anomaly classification and selected indicators, including failed connections, connection rate, and unusual-port activity.

The resulting score is mapped to one of four categories: Critical, High, Medium, or Low.

This component makes the output easier to interpret, but the risk categories remain on project-defined classifications rather than standardized cybersecurity severity levels.

6.7 Results and Visualization:

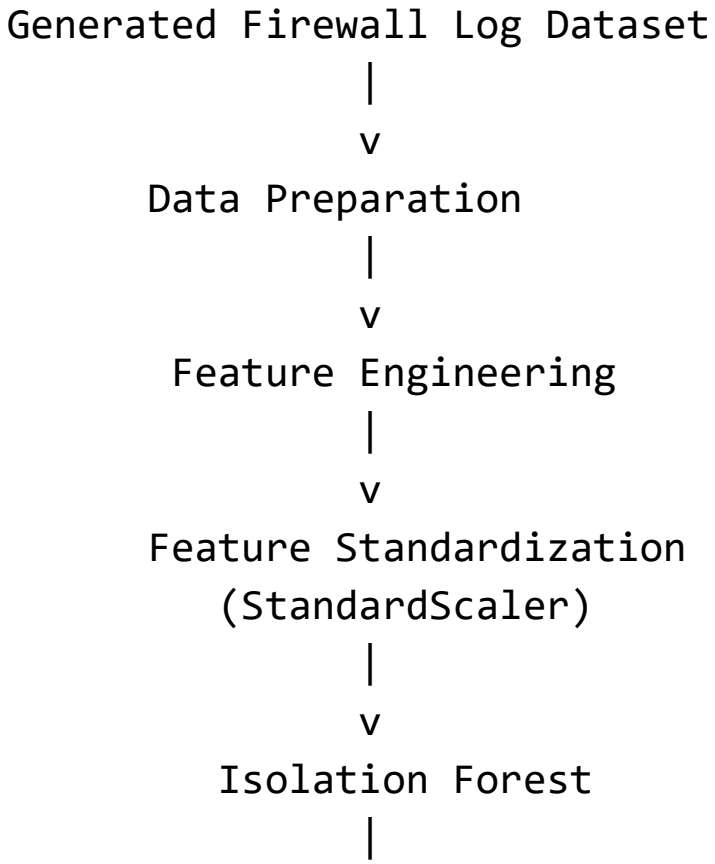
The final stage summarizes the detection results and presents them in a human-readable form.

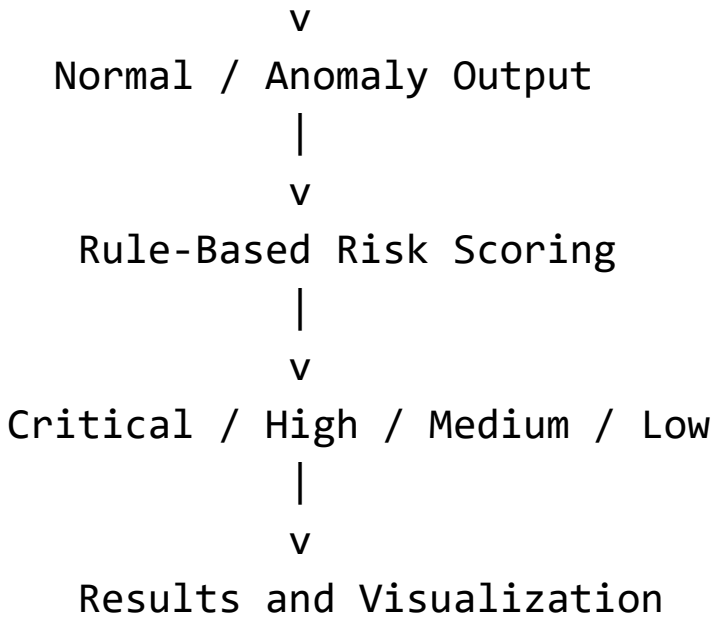
Possible outputs include the total number of records, normal and anomalous counts, anomaly percentage, risk-category distribution, and relevant graphs.

These summaries help the user examine the output of the detection pipeline. They should be interpreted alongside the dataset's limitations and the distinction between anomalous behavior and confirmed malicious activity.

6.8 Overall Architecture Diagram:

The architecture can be represented as follows:





This diagram represents the main logical stages of the project implemented. It distinguishes the machine-learning detection stage from the rule-based scoring stage and avoids implying that the system confirms attacks or automatically responds to security incidents.

7. Technologies and Tools

7.1 Python:

Python is a high-level programming language widely used for data analysis, automation, scientific computing, and machine learning.

Its extensive library ecosystem makes it suitable for processing structured firewall data, deriving numerical features, applying anomaly detection algorithms, and generating visualizations.

In this project, Python provides the main programming environment for implementing the detection pipeline.

7.2 Pandas:

Pandas is a Python library for data manipulation and analysis. Its DataFrame structure represents data in rows and columns, making it useful for working with structured log records.

Within this project, Pandas supports data organization, feature manipulation, and the analysis of detection results.

7.3 NumPy:

NumPy provides numerical arrays and mathematical operations for scientific computing in Python.

It can support numerical calculations and transformations required by data-analysis workflows. Its inclusion in the project's active tool list should reflect its actual use in the notebook.

7.4 Scikit-learn:

Scikit-learn is a Python machine-learning library that provides tools for preprocessing, model fitting, prediction, and model evaluation.

Two scikit-learn components are particularly relevant to this project:

- Isolation Forest: Identifies observations that differ from the general pattern of the input data.
- StandardScaler: standardizes the selected numerical features before they are passed to the model.

Together, these components support the machine-learning stage of the detection pipeline.

7.5 Matplotlib:

Matplotlib is a Python library for creating graphs and visualizations. It can be used to present numerical results in a form that is easier to interpret.

In this project, it supports the visualization of detection summaries and other relevant statistics produced by the notebook.

7.6 Seaborn:

Seaborn is a statistical visualization library built on top of Matplotlib. It provides functions for creating statistical charts and other graphical summaries.

Its role in the project depends on the visualizations actually generated in the notebook. It should be documented as an implemented tool only when the code uses it.

7.7 Google Colab:

Google Colab is a cloud-based environment for executing Python notebooks through a web browser. It allows code, explanatory text, data processing, and visualizations to be organized within a single document.

It is suitable for this project because it supports interactive experimentation without requiring a complete local Python development environment.

7.8 Summary of Technologies:

Technology	Role in the project
Python	Main programming language
Pandas	Structured data manipulation and analysis
NumPy	Numerical operations, where used
Scikit-learn	Feature standardization and anomaly detection
Isolation Forest	Unsupervised anomaly-detection algorithm
StandardScaler	Standardization of numerical features
Matplotlib	Data visualization
Seaborn	Statistical visualization, where used
Google Colab	Cloud-based notebook development and execution

Table 6: Summary of technologies, libraries, and their roles in the project

The final documentation should reflect the libraries and components actually imported or used in the notebook. This improves technical accuracy and helps readers reproduce the implementation.

8. Conclusion

This theoretical study has introduced the main cybersecurity concepts underlying the Firewall Log Anomaly Detector. It has explained the role of firewalls and their logs, the nature of network anomalies, and the main approaches used to identify potentially unusual activity.

The study has also presented the theoretical principles of Isolation Forest, the importance of feature engineering, and the role of rule-based risk scoring. These concepts provide the foundation for understanding how the project transforms structured firewall data into anomaly classifications and risk categories.

The implementation combines unsupervised machine learning with predefined scoring conditions. Isolation Forest identifies observations that differ from patterns in the dataset, while the risk-scoring mechanism assigns interpretable categories based on selected indicators. This distinction is essential for understanding the system's output and its limitations.

The project should be understood as a prototype for automated firewall log analysis. Its results depend on the quality of the input data, the selected features, the model configuration, and the scoring rules. Further testing with representative real-world datasets and reliable ground-truth labels would be necessary to assess its effectiveness in practical cybersecurity environments.

The second part of this report presents the practical implementation, including the Python environment, dataset preparation, feature engineering, Isolation Forest configuration, risk-scoring logic, visualizations, and obtained results.

Part II — Practical Part

1. Introduction

This chapter describes the implementation of the Firewall Log Anomaly Detector in Google Colab. The project was designed as a prototype that processes firewall-like records, identifies observations that differ from common traffic patterns, assigns rule-based risk levels, and generates readable security alerts. The workflow also includes functional tests on new records and the export of the trained model and processed dataset.

Because real organizational firewall logs were not available, the experiment uses a synthetic dataset created in Python. The results therefore demonstrate the operation of the pipeline on controlled examples; they do not establish detection accuracy on real cyberattacks or production network traffic.

Implementation workflow: **Synthetic data generation → exploratory data analysis → feature engineering → scaling → Isolation Forest → anomaly review → risk scoring → alert generation → functional testing and export.**

2. Development Environment and Dataset Preparation

a. Development environment and libraries:

Google Colab was used to develop and execute the Python notebook without requiring a local machine-learning setup. Pandas and NumPy were used for data manipulation and numerical operations. Matplotlib and Seaborn supported exploratory visualization. Scikit-learn provided StandardScaler and Isolation Forest, while Joblib was used to save the trained model and preprocessing object.

```
print("Firewall Log Anomaly Detector")
print("Practical implementation started successfully.")
```

```
Firewall Log Anomaly Detector
Practical implementation started successfully.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

import seaborn as sns

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

Figure 1: Environment setup and library imports for the firewall anomaly detector

b. Generating synthetic firewall records:

The dataset was generated directly in the notebook. It contains 5,000 simulated normal-traffic records and 250 simulated anomalous records, for a total of 5,250 records. Normal records were designed to have generally lower packet counts, smaller data volumes, fewer failed connections, and moderate connection rates. The anomalous examples were assigned more extreme traffic values and destination ports selected for this experiment. A fixed random seed was used to make the data-generation process reproducible.

```
np.random.seed(42)
n_normal = 5000
n_anomalies = 250
normal_data = pd.DataFrame({
    "src_port": np.random.randint(1024, 50000, n_normal),

    "dst_port": np.random.choice(
        [80, 443, 22, 53, 25, 110, 143],
        n_normal ),
    "protocol": np.random.choice(
        ["TCP", "UDP"],
        n_normal,
        p=[0.75, 0.25]  ),

    "duration": np.random.exponential(
        2,
        n_normal  ),

    "packets": np.random.poisson(
        20,
        n_normal  ),

    "bytes": np.random.normal(
        5000,
        1500,
        n_normal  ),

    "failed_connections": np.random.poisson(
        0.2,
        n_normal    ),

    "connection_rate": np.random.normal(
        5,
        1.5,
        n_normal )
})
```

Figure 2: Generation of synthetic normal firewall traffic records

```
normal_data["bytes"] = normal_data["bytes"].clip(lower=100)
anomaly_data = pd.DataFrame({
    "src_port": np.random.randint(
        1,
        65535,
        n_anomalies  ),

    "dst_port": np.random.choice(
        [21, 23, 445, 3389, 4444, 5555, 8080, 9999],
        n_anomalies  ),

    "protocol": np.random.choice(
        ["TCP", "UDP"],
        n_anomalies ),

    "duration": np.random.uniform(
        30,
        300,
        n_anomalies ),

    "packets": np.random.randint(
        200,
        2000,
        n_anomalies ),

    "bytes": np.random.randint(
        50000,
        1000000,
        n_anomalies  ),

    "failed_connections": np.random.randint(
        5,
        100,
        n_anomalies  ),
```

Figure 3: Generation of synthetic anomalous traffic and clipping pipeline

```
    "connection_rate": np.random.uniform(
        50,
        500,
        n_anomalies  )
})

df = pd.concat(
    [normal_data, anomaly_data],
    ignore_index=True
)
df = df.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

print("Firewall dataset created successfully!")
print()
print(f"Normal traffic records   : {n_normal}")
print(f"Anomalous records        : {n_anomalies}")
print(f"Total records            : {len(df)}")
```

Figure 4: Dataset merging, shuffling, and class distribution verification

c. Adding timestamps and IP addresses:

Synthetic source and destination IP addresses were added to make the records resemble firewall log entries. A timestamp was also generated for each record, beginning on 1 January 2026 and advancing at one-minute

intervals. These addresses and timestamps are simulated metadata; they do not represent observed network activity.

```python
def generate_ip():
    return ".".join(
        str(np.random.randint(1, 255))
        for _ in range(4)
    )
df["src_ip"] = [
    generate_ip()
    for _ in range(len(df))
]

df["dst_ip"] = [
    generate_ip()
    for _ in range(len(df))
]

df["timestamp"] = pd.date_range(
    start="2026-01-01",
    periods=len(df),
    freq="min"
)

df = df[
    [
        "timestamp",
        "src_ip",
        "dst_ip",
        "src_port",
        "dst_port",
        "protocol",
        "duration",
        "packets",
        "bytes",
        "failed_connections",
        "connection_rate"
    ]
]
```

Figure 5: IP address generation, timestamp injection, and dataset feature reordering

3. Exploratory Data Analysis

a. Inspecting the dataset:

The first rows, dataset dimensions, column types, and non-null counts were inspected before building the model. This step confirmed that the generated dataset contained 5,250 rows and 11 columns covering timestamps, IP

addresses, ports, protocols, connection duration, packet and byte counts, failed connections, and connection rate.

```
df.head(10)
```

	timestamp	src_ip	dst_ip	src_port	dst_port	protocol	duration	packets	bytes	failed_connections	connection_rate
0	2026-01-01 00:00:00	222.18.13.14	137.247.32.72	22816	22	TCP	0.777311	12	7709.963375	0	4.963247
1	2026-01-01 00:01:00	41.40.249.89	151.27.42.8	27213	143	TCP	4.515295	21	5867.563682	0	5.683014
2	2026-01-01 00:02:00	101.219.215.199	221.11.149.243	1364	110	UDP	2.190574	16	5155.006333	0	4.812960
3	2026-01-01 00:03:00	48.10.217.121	44.167.222.33	49543	143	UDP	0.618634	26	4853.359653	0	6.721246
4	2026-01-01 00:04:00	54.163.152.49	67.55.13.36	35075	25	TCP	1.489982	18	4294.237692	1	4.927919
5	2026-01-01 00:05:00	170.203.84.202	87.26.202.38	22986	110	UDP	3.423835	13	3401.409897	0	4.106809
6	2026-01-01 00:06:00	50.149.156.220	100.151.136.14	45044	25	TCP	0.409792	15	6404.356552	0	5.229848
7	2026-01-01 00:07:00	183.155.97.68	24.119.210.168	28504	110	TCP	0.107840	18	3807.715868	0	4.846795
8	2026-01-01 00:08:00	99.84.134.186	101.238.153.118	3787	22	TCP	4.731212	21	4604.929477	0	5.785476
9	2026-01-01 00:09:00	134.60.81.147	239.212.153.17	39583	53	TCP	3.469887	16	5081.534292	0	7.710393

```
print("Dataset dimensions:")
print(df.shape)
```

```
Dataset dimensions:
(5250, 11)
```

Figure 6: Preview of the generated firewall dataset and structural dimensions

```
print("Dataset information:")
print()

df.info()
```

```
Dataset information:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5250 entries, 0 to 5249
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   timestamp           5250 non-null   datetime64[ns]
 1   src_ip              5250 non-null   object
 2   dst_ip              5250 non-null   object
 3   src_port            5250 non-null   int64
 4   dst_port            5250 non-null   int64
 5   protocol            5250 non-null   object
 6   duration            5250 non-null   float64
 7   packets             5250 non-null   int64
 8   bytes               5250 non-null   float64
 9   failed_connections  5250 non-null   int64
 10  connection_rate     5250 non-null   float64
dtypes: datetime64[ns](1), float64(3), int64(4), object(3)
memory usage: 451.3+ KB
```

Figure 7: Dataset schema, data types, and null-value verification

b. Data quality and traffic distributions:

Missing values and descriptive statistics were checked to understand the range of numerical variables. No missing values were present in the generated columns. The normal-traffic connection rate was sampled from a normal distribution, which can produce negative values; the code therefore clipped this variable at zero because negative connection rates are not meaningful in this simulation.

```
print("Missing values in each column:")
print()

print(df.isnull().sum())
```

```
Missing values in each column:

timestamp             0
src_ip                0
dst_ip                0
src_port              0
dst_port              0
protocol              0
duration              0
packets               0
bytes                 0
failed_connections    0
connection_rate       0
dtype: int64
```

Figure 8: Missing values verification

```
print("Statistical Summary:")
print()

df.describe()
```

```
Statistical Summary:
```

	timestamp	src_port	dst_port	duration	packets	bytes	failed_connections	connection_rate
count	5250	5250.000000	5250.000000	5250.000000	5250.000000	5250.000000	5250.000000	5250.000000
mean	2026-01-02 19:44:29.999999744	25613.363429	311.328571	9.995322	69.970476	30882.192112	2.732000	18.073478
min	2026-01-01 00:00:00	176.000000	21.000000	0.000237	6.000000	100.000000	0.000000	-0.325254
25%	2026-01-01 21:52:15	13030.000000	25.000000	0.625741	17.000000	4033.713471	0.000000	4.076985
50%	2026-01-02 19:44:30	25446.000000	80.000000	1.507544	20.000000	5093.563425	0.000000	5.124671
75%	2026-01-03 17:36:45	37757.000000	143.000000	3.114911	24.000000	6204.772532	0.000000	6.188946
max	2026-01-04 15:29:00	65335.000000	9999.000000	299.871055	1987.000000	988054.000000	99.000000	496.296645
std	NaN	14428.608721	1120.321295	39.703705	250.401978	129601.340248	12.683448	65.018888

Figure 9: Statistical summary of firewall log numerical features

```
df["connection_rate"] = df["connection_rate"].clip(lower=0)

print("Connection rate corrected successfully.")
print(f"Minimum connection rate: {df['connection_rate'].min():.2f}")
```

```
Connection rate corrected successfully.
Minimum connection rate: 0.00
```

Figure 10: Connection rate data correction

The protocol counts, destination-port frequencies, and packet-count distribution were then visualized. TCP was more frequent than UDP because normal records were generated using a 75%/25% TCP-to-UDP sampling probability. The port chart also reflects the design of the synthetic dataset: common service ports occur frequently in the simulated normal traffic, whereas the selected ports used for unusual records occur less often overall.

```
print("Protocol distribution:")
print()

print(df["protocol"].value_counts())
```

```
Protocol distribution:

protocol
TCP    3903
UDP    1347
Name: count, dtype: int64
```

```
print("Most common destination ports:")
print()

print(
    df["dst_port"].value_counts().head(15)
)
```

```
Most common destination ports:

dst_port
443     766
143     744
25      725
110     712
53      698
22      689
80      666
4444     37
21       34
5555     34
8080     33
445      32
23       29
9999     27
3389     24
Name: count, dtype: int64
```

Figure 11: Exploratory data analysis of protocol distribution and destination port frequencies

```
port_counts = df["dst_port"].value_counts().head(15)
plt.figure(figsize=(12, 6))
port_counts.plot( kind="bar")
plt.title("Most Common Destination Ports")
plt.xlabel("Destination Port")
plt.ylabel("Number of Connections")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

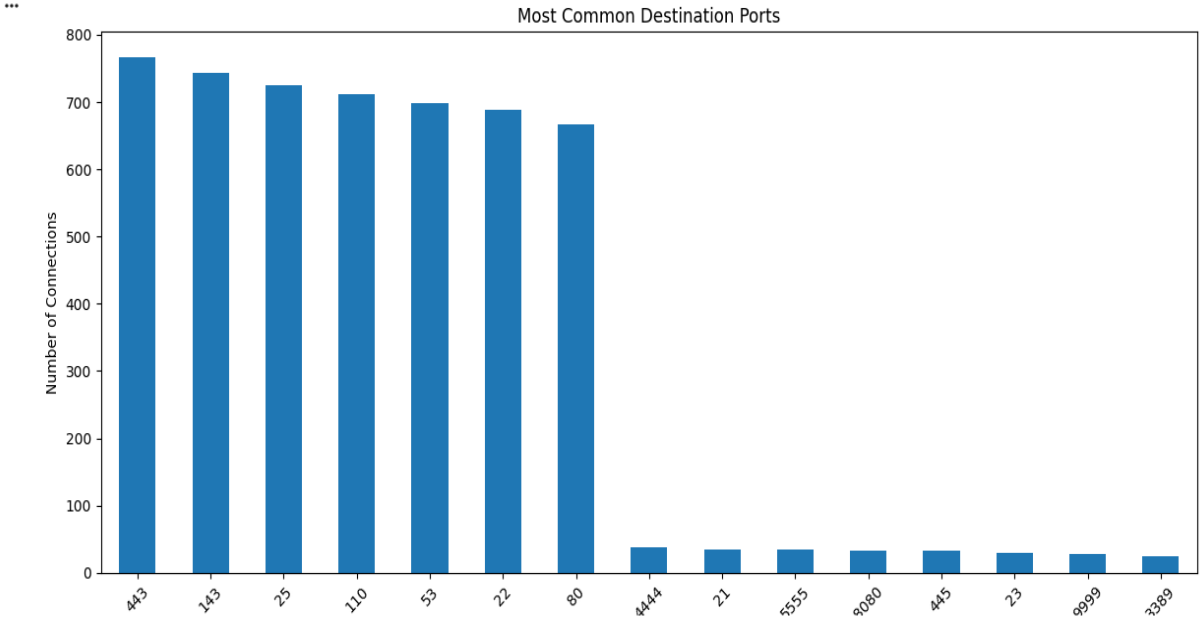


Figure 12: Bar chart visualization of the most common destination ports

```
plt.figure(figsize=(10, 6))
plt.hist( df["packets"],
    bins=50)
plt.title("Distribution of Network Packets")
plt.xlabel("Number of Packets")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

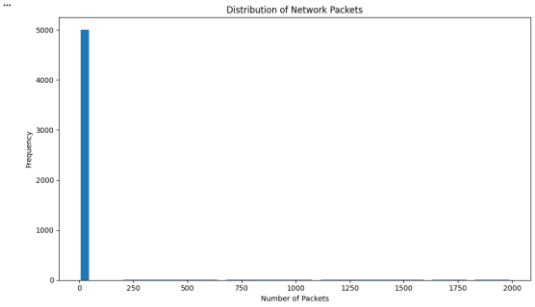


Figure 13: Histogram visualization of network packet distribution

4. Feature Engineering and Preprocessing

a. Creating derived features:

Feature engineering was applied to derive additional indicators from the raw log fields. Five features were added: the hour of the event, bytes per packet, packet rate, a failed-connection indicator divided by the connection-rate value, and a binary flag for destination ports outside a selected list of common ports. These features provide the model with both traffic-volume measurements and simple indicators of unusual patterns.

```
df["hour"] = df["timestamp"].dt.hour
df["bytes_per_packet"] = (
    df["bytes"] / df["packets"].replace(0, 1))
df["packet_rate"] = (
    df["packets"] / df["duration"].replace(0, 1))
df["failed_connection_ratio"] = (
    df["failed_connections"]
    / df["connection_rate"].replace(0, 1))

common_ports = [22, 25, 53, 80, 110, 143, 443]
df["unusual_port"] = (
    ~df["dst_port"].isin(common_ports)
).astype(int)

print("Feature engineering completed successfully!")
print()
print("New features:")
print([
    "hour",
    "bytes_per_packet",
    "packet_rate",
    "failed_connection_ratio",
    "unusual_port"
])
```

```
Feature engineering completed successfully!

New features:
['hour', 'bytes_per_packet', 'packet_rate', 'failed_connection_ratio', 'unusual_port']
```

Figure 14: Feature engineering and derived network metrics creation

Important interpretation: The feature named failed_connection_ratio is calculated as failed_connections divided by connection_rate in this notebook. It should be interpreted as a simplified indicator, not a true percentage of failed connection attempts. A future version should use the total number of connection attempts as the denominator, if available.

b. Selecting model inputs and scaling:

Twelve numerical features were selected for training. The protocol field was included in exploratory analysis but was not encoded as a model input. The selected feature matrix was transformed with StandardScaler, and the fitted scaler was saved for reuse when assessing new records. Applying the same transformation to future inputs is important because the model expects features in the same representation as those used during training.

```
print("Engineered features:")
print()
print(
    df[
        [
            "hour",
            "bytes_per_packet",
            "packet_rate",
            "failed_connection_ratio",
            "unusual_port"
        ]
    ].head(10)
)
```

```
Engineered features:

   hour  bytes_per_packet  packet_rate  failed_connection_ratio  unusual_port
0     0        642.496948    15.437837                 0.000000             0
1     0        279.407794     4.650859                 0.000000             0
2     0        322.187896     7.304022                 0.000000             0
3     0        186.667679    42.028082                 0.000000             0
4     0        238.568761    12.080686                 0.202925             0
5     0        261.646915     3.796911                 0.000000             0
6     0        426.957103    36.603921                 0.000000             0
7     0        211.539770   166.913659                 0.000000             0
8     0        219.282356     4.438609                 0.000000             0
9     0        317.595893     4.611101                 0.000000             0
```

Figure 15: Preview of engineered features and derived metrics

```
features = [
    "src_port",
    "dst_port",
    "duration",
    "packets",
    "bytes",
    "failed_connections",
    "connection_rate",
    "hour",
    "bytes_per_packet",
    "packet_rate",
    "failed_connection_ratio",
    "unusual_port"
]

X = df[features]

print("Machine learning features selected successfully!")
print()
print(f"Number of features: {X.shape[1]}")
print(f"Number of records: {X.shape[0]}")
print()
print("Selected features:")
print(features)
```

```
Machine learning features selected successfully!

Number of features: 12
Number of records: 5250

Selected features:
['src_port', 'dst_port', 'duration', 'packets', 'bytes', 'failed_connections', 'connection_rate', 'hour', 'bytes_per_packet', 'packet_rate', 'failed_connection_ratio', 'unusual_port']
```

Figure 16: Selection of machine learning features and dataset shape verification

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("Feature scaling completed successfully!")
print()
print(f"Scaled dataset shape: {X_scaled.shape}")
```

```
Feature scaling completed successfully!

Scaled dataset shape: (5250, 12)
```

Figure 17: Feature scaling and standardization using StandardScaler

5. Training and Applying the Isolation Forest Model

a. Model training:

Isolation Forest was selected as the unsupervised anomaly detection algorithm. It builds randomized isolation trees and identifies observations

that can be separated from the wider dataset relatively easily. In this experiment, the model was fitted to the scaled features without supplying ground-truth labels to the learning algorithm.

```
model = IsolationForest(
    n_estimators=200,
    contamination=0.05,
    random_state=42
)

model.fit(X_scaled)

print("Isolation Forest model trained successfully!")
print()
print("Number of trees:", model.n_estimators)
print("Expected anomaly proportion:", model.contamination)
```

```
Isolation Forest model trained successfully!

Number of trees: 200
Expected anomaly proportion: 0.05
```

Figure 18: Isolation Forest model initialization, hyperparameter configuration, and training

Parameter clarification: n_estimators=200 specifies 200 isolation trees. Contamination=0.05 configures the model to use an expected anomaly proportion of approximately 5% when establishing its decision threshold. It is not an accuracy score and does not prove that 5% of real network traffic is malicious.

b. Predicting and summarizing anomalies:

The trained model was applied to the same feature matrix. Isolation Forest returns 1 for records classified as normal and -1 for records classified as anomalies. These numeric outputs were converted into readable labels and summarized.

```
df["anomaly"] = model.predict(X_scaled)
df["anomaly_label"] = df["anomaly"].map({
    1: "Normal",
    -1: "Anomaly"
})

print("Anomaly detection completed successfully!")
print()
print(df["anomaly_label"].value_counts())
```

```
Anomaly detection completed successfully!

anomaly_label
Normal     4987
Anomaly     263
Name: count, dtype: int64
```

```
total_records = len(df)
anomaly_count = (df["anomaly"] == -1).sum()
normal_count = (df["anomaly"] == 1).sum()

anomaly_percentage = (
    anomaly_count / total_records
) * 100

print("Anomaly Detection Summary")
print("=" * 35)
print(f"Total firewall logs : {total_records}")
print(f"Normal logs         : {normal_count}")
print(f"Anomalous logs      : {anomaly_count}")
print(f"Anomaly percentage  : {anomaly_percentage:.2f}%")
```

```
Anomaly Detection Summary
===================================
Total firewall logs : 5250
Normal logs         : 4987
Anomalous logs      : 263
Anomaly percentage  : 5.01%
```

Figure 19: Anomaly prediction, labeling, and detection summary metrics

The model marked 263 records as anomalous, corresponding to 5.01% of the dataset. This result is close to the configured contamination value, as expected. The output should be understood as a list of records that differ from the modelled pattern, not as a confirmation that those records represent actual attacks.

6. Anomaly Review, Risk Scoring, and Alert Generation

a. Inspecting detected records:

The records labelled as anomalies were extracted and reviewed using their timestamp, addresses, ports, protocol, duration, packet count, data volume, failed connections, and connection rate. Several flagged examples combine high packet counts, large data volumes, elevated connection rates, and multiple failed connections. The model evaluates the combined pattern; one field, such as a destination port, is not sufficient on its own to establish malicious activity.

```
anomalies = df[df["anomaly"] == -1]

print("Detected Anomalous Firewall Logs")
print("=" * 40)
print(f"Number of anomalies: {len(anomalies)}")
print()
anomalies[
    [
        "timestamp",
        "src_ip",
        "dst_ip",
        "src_port",
        "dst_port",
        "protocol",
        "duration",
        "packets",
        "bytes",
        "failed_connections",
        "connection_rate"
    ]
].head(10)
```

```
Detected Anomalous Firewall Logs
========================================
Number of anomalies: 263
```

	timestamp	src_ip	dst_ip	src_port	dst_port	protocol	duration	packets	bytes	failed_connections	connection_rate
26	2026-01-01 00:26:00	140.124.177.84	58.220.18.172	8928	3389	TCP	74.820141	590	555236.00000	81	423.367371
29	2026-01-01 00:29:00	160.101.79.52	206.247.110.166	49450	21	TCP	77.094462	328	305460.00000	43	224.732279
31	2026-01-01 00:31:00	220.30.182.235	128.10.63.94	14716	8080	TCP	269.499170	1866	604472.00000	15	313.004022
67	2026-01-01 01:07:00	10.162.146.53	197.69.48.248	41393	5555	TCP	133.982301	452	539712.00000	69	201.782763
110	2026-01-01 01:50:00	84.218.238.47	61.106.235.213	65194	3389	TCP	47.147920	480	440311.00000	65	262.987514
115	2026-01-01 01:55:00	147.67.216.151	18.5.118.216	42455	445	TCP	141.821534	1744	715151.00000	61	262.975725
124	2026-01-01 02:04:00	198.2.207.121	187.151.254.228	2366	443	TCP	0.018303	22	6338.28297	2	5.851020
201	2026-01-01 03:21:00	148.204.114.85	212.193.76.227	63878	4444	TCP	269.084519	1672	931872.00000	27	119.088845
214	2026-01-01 03:34:00	24.98.41.239	113.237.94.253	19787	21	TCP	196.256278	936	485637.00000	88	463.659072
226	2026-01-01 03:46:00	137.247.169.50	41.122.157.143	19492	9999	UDP	286.102873	1839	393607.00000	85	405.959556

Figure 20: Extraction and preview of detected anomalous firewall logs

b. Exploring anomaly patterns:

The destination ports and protocol values among detected records were counted, and scatter plots were produced to explore relationships between packet count and byte volume, as well as between connection rate and failed connections. The most frequent ports included 4444, 5555, 21, 8080, 445, 23, 9999, and 3389. These results must be interpreted in the context of the synthetic data design, which deliberately used these ports for anomalous examples.

```
anomaly_ports = anomalies["dst_port"].value_counts()

print("Destination Ports in Detected Anomalies")
print("=" * 40)
print()

print(anomaly_ports)
```

```
Destination Ports in Detected Anomalies
========================================

dst_port
4444    37
5555    34
21      34
8080    33
445     32
23      29
9999    27
3389    24
443      6
25       4
53       1
22       1
143      1
Name: count, dtype: int64
```

Figure 21: Destination port frequency distribution across detected anomalies

```
plt.figure(figsize=(10, 6))
plt.scatter(
    anomalies["packets"],
    anomalies["bytes"],
    alpha=0.7)
plt.title("Detected Anomalies: Packets vs Bytes")
plt.xlabel("Number of Packets")
plt.ylabel("Bytes")
plt.tight_layout()
plt.show()
```

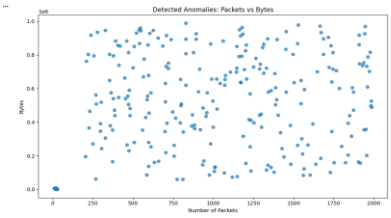


Figure 22: Scatter plot visualization of packet count versus byte volume in detected anomalies

```
plt.figure(figsize=(10, 6))

plt.scatter(
    anomalies["connection_rate"],
    anomalies["failed_connections"],
    alpha=0.7
)

plt.title("Detected Anomalies: Connection Rate vs Failed Connections")
plt.xlabel("Connection Rate")
plt.ylabel("Failed Connections")

plt.tight_layout()
plt.show()
```

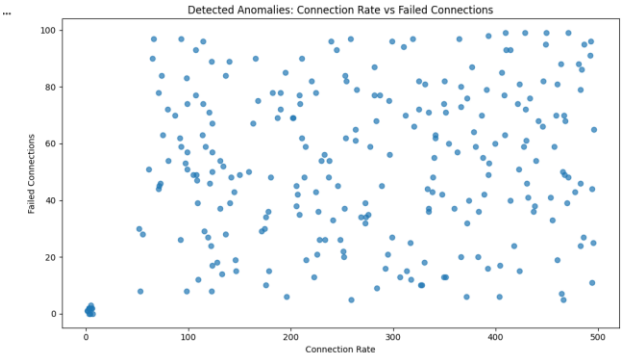


Figure 23: Scatter plot visualization of connection rate versus failed connections in detected anomalies

c. Rule-based risk score:

To make model outputs easier to prioritize, a second layer assigned points according to four rules: 50 points for a model anomaly, 20 points for more than ten failed connections, 15 points for a connection rate above 50, and 15 points for a destination port outside the selected common-port list. The score was capped at 100.

```
df["risk_score"] = 0

df.loc[df["anomaly"] == -1, "risk_score"] += 50

df.loc[df["failed_connections"] > 10, "risk_score"] += 20

df.loc[df["connection_rate"] > 50, "risk_score"] += 15

df.loc[df["unusual_port"] == 1, "risk_score"] += 15

df["risk_score"] = df["risk_score"].clip(upper=100)

print("Risk scoring completed successfully!")
print()
print("Risk score statistics:")
print(df["risk_score"].describe())
```

```
Risk scoring completed successfully!

Risk score statistics:
count    5250.000000
mean        4.836190
std        21.217061
min         0.000000
25%         0.000000
50%         0.000000
75%         0.000000
max       100.000000
Name: risk_score, dtype: float64
```

Figure 24: Risk scoring implementation and statistical distribution analysis

How to describe this score? It is a manually defined prioritization score that combines the anomaly label with simple thresholds. It is not a probability of an attack and has not been calibrated against labelled real-world traffic.

d. Assigning risk levels:

```python
def classify_risk(score):
    if score >= 80:
        return "Critical"
    elif score >= 60:
        return "High"
    elif score >= 30:
        return "Medium"
    else:
        return "Low"

df["risk_level"] = df["risk_score"].apply(classify_risk)

print("Risk classification completed successfully!")
print()
print("Risk level distribution:")
print()
print(df["risk_level"].value_counts())
```

```
Risk classification completed successfully!

Risk level distribution:

risk_level
Low         4987
Critical     250
Medium        13
Name: count, dtype: int64
```

Figure 25: Risk level classification logic and category distribution analysis

```python
risk_counts = df["risk_level"].value_counts()
plt.figure(figsize=(8, 6))
risk_counts.plot(
    kind="bar"
)
plt.title("Firewall Risk Level Distribution")
plt.xlabel("Risk Level")
plt.ylabel("Number of Logs")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

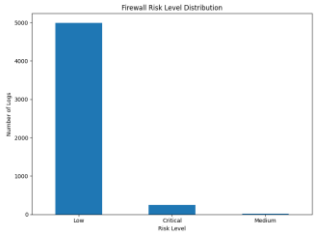


Figure 26: Bar chart visualization of firewall risk level distribution

Risk level	Records
Critical	250
High	0
Medium	13
Low	4,987
Total	5,250

Table 7: Quantitative distribution of firewall logs by risk level

In this experiment, 250 records were assigned Critical risk and 13 were assigned medium risk; no records were assigned High risk. The 250 Critical records correspond to the deliberately generated anomalous traffic patterns. Thirteen other records were flagged by Isolation Forest but received a medium score under the manually defined rules. A Low label only means that these rules did not assign a higher level; it is not a guarantee that a record is safe.

e. Generating security alerts:

A function converted the risk level into a readable alert message. Critical, High, and Medium records receive a message that includes the source IP address, destination IP address, and destination port. Low-risk records receive a message stating that no alert was generated.

```python
def generate_alert(row):
    if row["risk_level"] == "Critical":
        return (
            f"CRITICAL ALERT: Suspicious traffic detected "
            f"from {row['src_ip']} to {row['dst_ip']} "
            f"on port {row['dst_port']}."
        )

    elif row["risk_level"] == "High":
        return (
            f"HIGH RISK ALERT: Unusual traffic detected "
            f"from {row['src_ip']} to {row['dst_ip']} "
            f"on port {row['dst_port']}."
        )

    elif row["risk_level"] == "Medium":
        return (
            f"MEDIUM RISK ALERT: Potentially unusual traffic "
            f"detected from {row['src_ip']} to {row['dst_ip']} "
            f"on port {row['dst_port']}."
        )

    else:
        return "No security alert."


df["security_alert"] = df.apply(
    generate_alert,
    axis=1
)
```

Figure 27: Implementation of the automated security alert generation function

```
print("Security alert generation completed successfully!")
print()

print("Example security alerts:")
print()

print(
    df[df["risk_level"] != "Low"][
        [
            "timestamp",
            "src_ip",
            "dst_ip",
            "dst_port",
            "risk_score",
            "risk_level",
            "security_alert"
        ]
    ].head(10).to_string(index=False)
)
```

```
Security alert generation completed successfully!

Example security alerts:

          timestamp         src_ip          dst_ip  dst_port  risk_score risk_level                                                                                                  security_alert
2026-01-01 00:26:00 140.124.177.84   58.220.18.172      3389         100   Critical            CRITICAL ALERT: Suspicious traffic detected from 140.124.177.84 to 58.220.18.172 on port 3389.
2026-01-01 00:29:00  160.101.79.52 206.247.110.166        21         100   Critical             CRITICAL ALERT: Suspicious traffic detected from 160.101.79.52 to 206.247.110.166 on port 21.
2026-01-01 00:31:00 220.30.182.235    128.10.63.94      8080         100   Critical             CRITICAL ALERT: Suspicious traffic detected from 220.30.182.235 to 128.10.63.94 on port 8080.
2026-01-01 01:07:00  10.162.146.53   197.69.48.248      5555         100   Critical             CRITICAL ALERT: Suspicious traffic detected from 10.162.146.53 to 197.69.48.248 on port 5555.
2026-01-01 01:50:00  84.218.238.47  61.106.235.213      3389         100   Critical            CRITICAL ALERT: Suspicious traffic detected from 84.218.238.47 to 61.106.235.213 on port 3389.
2026-01-01 01:55:00 147.67.216.151    18.5.118.216       445         100   Critical              CRITICAL ALERT: Suspicious traffic detected from 147.67.216.151 to 18.5.118.216 on port 445.
2026-01-01 02:04:00  198.2.207.121 187.151.254.228       443          50     Medium MEDIUM RISK ALERT: Potentially unusual traffic detected from 198.2.207.121 to 187.151.254.228 on port 443.
2026-01-01 03:21:00 148.204.114.85  212.193.76.227      4444         100   Critical           CRITICAL ALERT: Suspicious traffic detected from 148.204.114.85 to 212.193.76.227 on port 4444.
2026-01-01 03:34:00   24.98.41.239  113.237.94.253        21         100   Critical               CRITICAL ALERT: Suspicious traffic detected from 24.98.41.239 to 113.237.94.253 on port 21.
2026-01-01 03:46:00 137.247.169.50  41.122.157.143      9999         100   Critical           CRITICAL ALERT: Suspicious traffic detected from 137.247.169.50 to 41.122.157.143 on port 9999.
```

Figure 28: Example preview of generated security alert messages for non-low risk logs

The alerting function creates messages for review. It does not block connections, send notifications to an external system, or interact with a live firewall.

7. Functional Testing and Exporting the Results

a. Reusable function for new records:

A reusable function was implemented to evaluate one new simulated record. It derives the same features used during training, arranges them in the same column order, applies the fitted scaler, and returns the model prediction. The protocol argument is accepted for completeness but is not used in the prediction because protocol was not encoded as one of the model features.

```python
def detect_anomaly(
    src_port,
    dst_port,
    protocol,
    duration,
    packets,
    bytes_value,
    failed_connections,
    connection_rate,
    timestamp
):

    new_data = pd.DataFrame({
        "src_port": [src_port],
        "dst_port": [dst_port],
        "duration": [duration],
        "packets": [packets],
        "bytes": [bytes_value],
        "failed_connections": [failed_connections],
        "connection_rate": [connection_rate],
        "hour": [pd.to_datetime(timestamp).hour]
    })

    new_data["bytes_per_packet"] = (
        new_data["bytes"] /
        new_data["packets"].replace(0, 1)
    )

    new_data["packet_rate"] = (
        new_data["packets"] /
        new_data["duration"].replace(0, 1)
    )

    new_data["failed_connection_ratio"] = (
        new_data["failed_connections"] /
        new_data["connection_rate"].replace(0, 1)
    )



    new_data["unusual_port"] = (
        ~new_data["dst_port"].isin(common_ports)
    ).astype(int)

    new_X = new_data[features]

    new_X_scaled = scaler.transform(new_X)

    prediction = model.predict(new_X_scaled)[0]

    if prediction == -1:
        result = "Anomaly"
    else:
        result = "Normal"

    return result


print("Anomaly detection function created successfully!")
```

Figure 29: Implementation of the custom detect_anomaly inference function and model prediction pipeline

b. Testing two simulated cases:

Two new examples were passed to the function. The first represented relatively ordinary traffic to destination port 443, with a low packet count and no failed connections. The second used destination port 4444, a high packet count and data volume, many failed connections, and a high connection rate.

```
normal_result = detect_anomaly(
    src_port=24567,
    dst_port=443,
    protocol="TCP",
    duration=1.8,
    packets=18,
    bytes_value=5200,
    failed_connections=0,
    connection_rate=5.2,
    timestamp="2026-01-10 10:30:00"
)

print("Normal Traffic Test")
print("=" * 30)
print()
print("Prediction:", normal_result)
```

```
Normal Traffic Test
==============================

Prediction: Normal
```

Figure 30: Simulation and classification test for normal network traffic

```
suspicious_result = detect_anomaly(
    src_port=52341,
    dst_port=4444,
    protocol="TCP",
    duration=180,
    packets=1200,
    bytes_value=650000,
    failed_connections=65,
    connection_rate=250,
    timestamp="2026-01-10 22:45:00"
)

print("Suspicious Traffic Test")
print("=" * 30)
print()
print("Prediction:", suspicious_result)
```

```
Suspicious Traffic Test
==============================

Prediction: Anomaly
```

Figure 31: Simulation and classification test for suspicious or anomalous network traffic

Test case	Observed model output
Simulated normal traffic	Normal
Simulated suspicious traffic	Anomaly

Table 8: Summary of simulation test cases and observed model outputs

Both designed examples produced the expected outputs. These are functional checks only: two cases are not sufficient to calculate model accuracy, measure the false-positive rate, or demonstrate performance on a representative network dataset.

c. Saving the model and processed dataset:

The trained Isolation Forest model and fitted scaler were saved with Joblib. The full processed DataFrame was exported to CSV. The exported dataset

contains the original log fields, five engineered features, anomaly predictions, risk score, risk level, and generated alert message.

```
import joblib

joblib.dump(model, "firewall_isolation_forest.pkl")
joblib.dump(scaler, "firewall_scaler.pkl")

print("Model and scaler saved successfully!")
print()
print("Saved files:")
print("- firewall_isolation_forest.pkl")
print("- firewall_scaler.pkl")
```

```
Model and scaler saved successfully!

Saved files:
- firewall_isolation_forest.pkl
- firewall_scaler.pkl
```

Figure 32: Model serialization and artifact persistence using Joblib

```python
output_file = "firewall_anomaly_detection_results.csv"

df.to_csv(
    output_file,
    index=False
)

print("Analyzed firewall dataset exported successfully!")
print()
print("File:", output_file)
print("Number of records:", len(df))
print("Number of columns:", len(df.columns))
```

```
Analyzed firewall dataset exported successfully!

File: firewall_anomaly_detection_results.csv
Number of records: 5250
Number of columns: 21
```

Figure 33: Final dataset export and completion of the analysis pipeline

Exported file	Purpose
firewall_isolation_forest.pkl	Trained Isolation Forest model
firewall_scaler.pkl	Fitted feature scaler required to preprocess new records
firewall_anomaly_detection_results.csv	Processed logs with predictions, risk levels, and alerts

Table 9: Summary of exported model artifacts and processed dataset files

8. Summary and Limitations

a. Summary of results:

```
==================================================
FIREWALL LOG ANOMALY DETECTOR - FINAL RESULTS
==================================================

Total firewall logs       : 5250
Normal logs               : 4987
Detected anomalies        : 263
Anomaly percentage        : 5.01%

Risk Classification
------------------------------
Critical                  : 250
High                      : 0
Medium                    : 13
Low                       : 4987

Model
------------------------------
Algorithm                 : Isolation Forest
Number of trees           : 200
Contamination             : 5%

Testing
------------------------------
Normal traffic test       : Normal
Suspicious traffic test   : Anomaly

Exported files
------------------------------
firewall_isolation_forest.pkl
firewall_scaler.pkl
firewall_anomaly_detection_results.csv

==================================================
PROJECT IMPLEMENTATION COMPLETED
==================================================
```

Figure 34: Final execution summary of the Firewall Log Anomaly Detector project

Indicator	Observed result
Number of generated records	5,250
Model output: Normal	4,987 (94.99%)
Model output: Anomaly	263 (5.01%)
Critical risk level	250
High risk level	0
Medium risk level	13

Low risk level	4,987
Algorithm	Isolation Forest
Number of trees	200
Configured contamination	0.05 (5%)
Functional test outputs	Normal example → Normal; suspicious example → Anomaly

Table 10: Comprehensive final metrics and results summary of the firewall anomaly detection project

b. Limitations:

The results should be interpreted within the limits of the prototype:

The records are synthetic and were generated with deliberately different traffic characteristics; they do not represent a realistic distribution of all network behavior.

The 5% contamination setting influences the number of observations the model identifies as anomalous. The 5.01% result is not an accuracy measure.

The notebook does not retain an independent ground-truth label for formal evaluation. Precision, recall, F1-score, and false-positive rate were therefore not measured.

The risk score is based on manually defined thresholds and is not a calibrated probability of malicious activity.

Protocol is not used as an input feature, and port values are treated as numerical values rather than as explicitly encoded categories.

The prototype does not ingest live firewall data, send real notifications, or block network traffic.

9. Conclusion and future improvements

The practical work implemented an end-to-end prototype for identifying unusual patterns in firewall-like records. The workflow covered synthetic data generation, data inspection, feature engineering, scaling, unsupervised model training, anomaly review, risk prioritization, alert generation, testing on new examples, and export of the model and results. Isolation Forest classified 263 out of 5,250 records as anomalies, while the rule-based layer assigned 250 records to Critical risk and 13 to medium risk.

The experiment demonstrates the technical workflow, but it does not prove real-world detection effectiveness. The next steps would be to test the approach on a public labelled intrusion-detection dataset, preserve ground-truth labels, evaluate precision and recall, improve feature definitions, compare different algorithms, and build a monitoring interface for incoming logs.

> **Overall conclusion:** The project illustrates how Python and unsupervised machine learning can support the triage of unusual network records. Any operational deployment would require validation on representative data and review by a cybersecurity analyst.

List of Figures:

Figure 1: Synthetic firewall log dataset generation and initial inspection

Figure 2: Verification of dataset dimensions and data types

Figure 3: Missing value check across firewall log features

Figure 4: Statistical summary and distribution of numerical attributes

Figure 5: Value counts and frequency analysis of categorical protocol fields

Figure 6: Distribution of source and destination IP addresses

Figure 7: Frequency analysis of destination ports in network traffic

Figure 8: Bar chart visualization of connection status outcomes

Figure 9: Correlation matrix heatmap of numerical firewall log features

Figure 10: Distribution plot of packet counts per connection

Figure 11: Distribution plot of byte counts transferred

Figure 12: Feature engineering process for derived temporal and statistical metrics

Figure 13: Inspection of newly engineered features (hour, bytes_per_packet, packet_rate)

Figure 14: Verification of missing values following feature engineering

Figure 15: Selection of numerical feature columns for scaling and modeling

Figure 16: Application of StandardScaler for feature normalization

Figure 17: Verification of scaled feature statistical properties (mean and variance)

Figure 18: Initialization and configuration of the Isolation Forest model

Figure 19: Model training execution on the standardized firewall log dataset

Figure 20: Prediction output extraction and anomaly labeling

Figure 21: Overall distribution of normal versus anomalous predictions

Figure 22: Visualization of anomaly scores across network log records

Figure 23: Scatter plot of packet count versus byte count with highlighted anomalies

Figure 24: Time-series representation of anomaly occurrences over simulated hours

Figure 25: Implementation of the rule-based risk-scoring mechanism

Figure 26: Bar chart visualization of firewall risk level distribution

Figure 27: Automated security alert generation for critical and high-risk logs

Figure 28: Sample inspection of flagged critical-risk security incidents

Figure 29: Inference pipeline implementation for processing new unseen firewall logs

Figure 30: Test execution of the inference pipeline with sample network records

Figure 31: Simulation and classification test for suspicious or anomalous network traffic

Figure 32: Serialization and export of trained Isolation Forest model artifacts (.pkl)

Figure 33: Export of processed dataset and anomaly results to CSV format

Figure 34: Final execution summary of the Firewall Log Anomaly Detector project

List of Tables:

Table 1: Description of raw firewall log attributes and fields

Table 2: Comparative analysis of signature-based, rule-based, and Isolation Forest detection approaches

Table 3: Summary and purpose of engineered features derived from raw firewall logs

Table 4: Rule-based risk scoring conditions and assigned risk points

Table 5: Risk score thresholds and threat level classifications

Table 6: Summary of technologies, libraries, and their roles in the project

Table 7: Quantitative distribution of firewall logs by risk level

Table 8: Summary of simulation test cases and observed model outputs

Table 9: Summary of exported model artifacts and processed dataset files

Table 10: Comprehensive final metrics and results summary of the firewall anomaly detection project

References:

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Article 15. https://doi.org/10.1145/1541880.1541882

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. In *Proceedings of the Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

National Institute of Standards and Technology. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)* (Special Publication 800-94). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-94

Google colab link:
https://colab.research.google.com/drive/1PjqNbinaJWzNe0cBxy0eP-qr5yMMOAaI?usp=sharing

General conclusion

This project focused on exploring the application of Artificial Intelligence and machine learning in the field of cybersecurity, particularly through the detection of abnormal activities in firewall logs. Its main objective was to develop an intelligent anomaly detection system capable of analyzing network traffic information, learning patterns associated with normal network behavior, and identifying activities that deviate from these patterns. Through the implementation of the Isolation Forest algorithm, the project demonstrated how machine learning can contribute to strengthening network monitoring and identifying potentially suspicious activities.

The development process involved several important stages, including dataset preparation, data exploration, feature engineering, model training, anomaly identification, and performance evaluation. Particular attention was given to selecting relevant network traffic features and preparing the data to ensure that the model could analyze different aspects of network behavior. The obtained results demonstrated the ability of the system to distinguish between normal and anomalous records and classify detected activities according to different risk levels. The practical tests also illustrated how automated anomaly detection can facilitate the analysis of large volumes of firewall logs.

Beyond its technical implementation, this project highlighted the importance of proactive network security in an increasingly digital and interconnected world. Traditional monitoring approaches may require significant human effort when analyzing large amounts of network data, whereas intelligent systems can assist administrators by identifying unusual patterns that deserve further investigation. However, anomaly detection does not necessarily mean that every identified anomaly represents a confirmed

cyberattack. Human verification and additional security investigation remain necessary to reduce false positives and correctly interpret detected events.

Overall, this project provided valuable practical experience in combining Artificial Intelligence, machine learning, and network security. It demonstrated the potential of intelligent anomaly detection as an additional layer of protection for modern information systems. It also opened perspectives for future improvements, such as testing the model on real-world firewall logs, integrating real-time monitoring capabilities, and exploring more advanced detection techniques to improve the identification of emerging network threats.